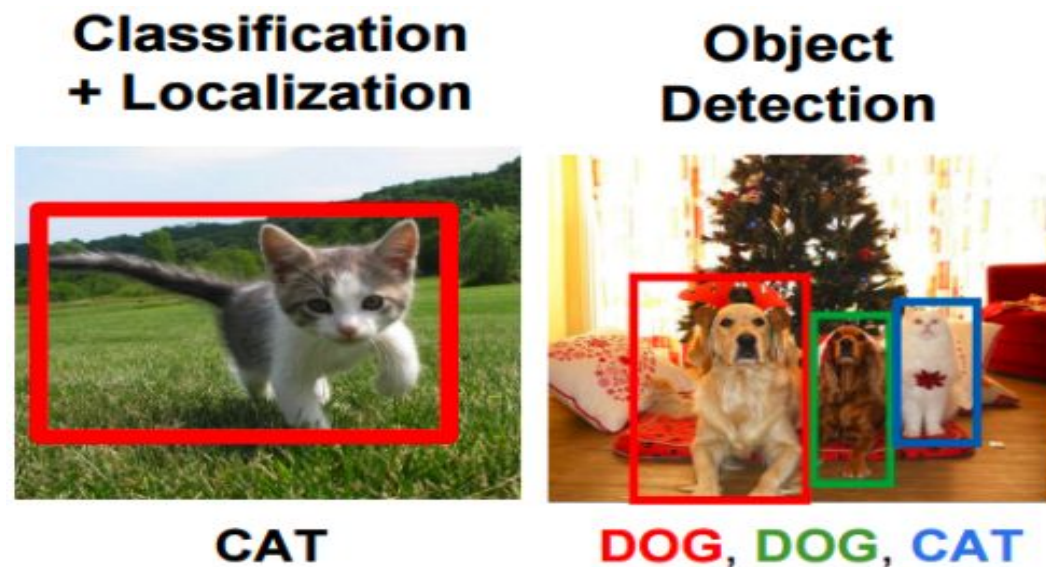


이미지 처리팀

Fast R-CNN
(Object Detection)

5523회 인용(구글 스칼라)

배경 지식

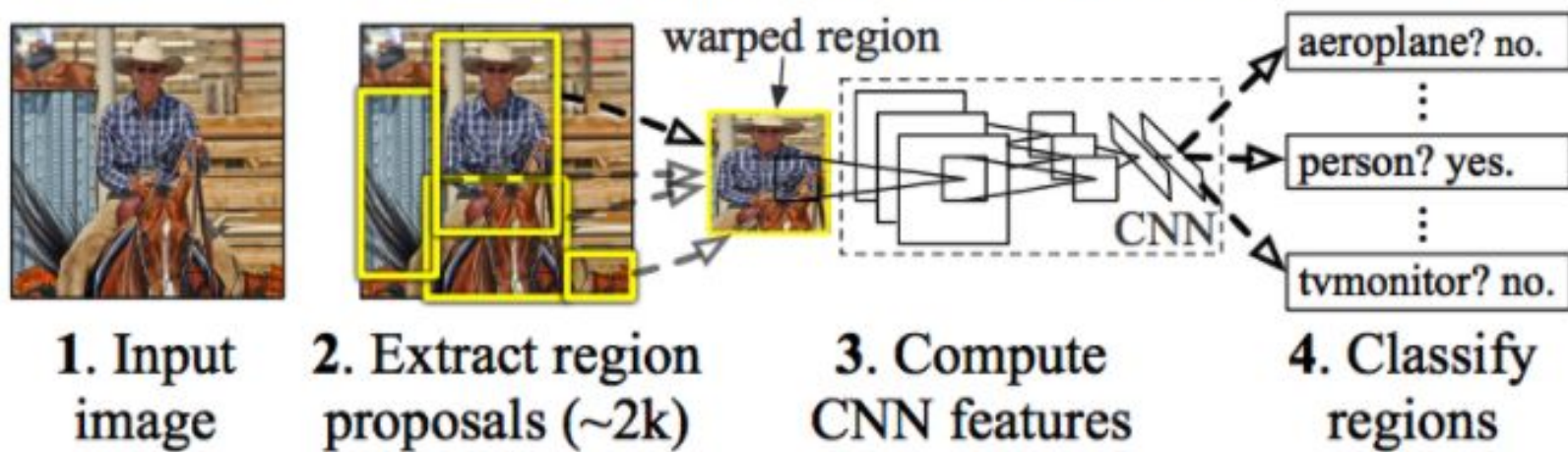


- Object detection이란 : 하나의 Image에서 여러 Object들을 검출하고 해당 Object가 어떤 Object 인지 Classifier를 통해 분류하는 기법
- Convolution layer의 이해

R-CNN

- 딥러닝을 이용한 가장 기본적인 Object detection 방법
- Input data는 Image로, 사람이 직접 boxing하고 어떤 Object인지 labeling해서 입력
- 기존 앙상블 시스템을 이용한(딥러닝을 이용하지 않은) 복잡한 Object detection보다 30% 성능 향상

R-CNN: *Regions with CNN features*



1. Selective Search(SS) 사용해 Object일 가능성이 있는 후보(proposal) 2,000개 검출
2. FC layer는 고정 크기 입력 받기에 각각의 proposal들을 227x227 size로 resizing
3. Proposal들을 각각 CNN에 통과시켜 feature map 벡터를 추출
4. 추출된 벡터를 classifier에 넣어 classification
5. Classification된 결과와 SS가 boxing한 결과를 back-propagation 통해 학습

R-CNN 의 한계

- 2,000개의 Proposal들을 각각 CNN에 넣으니 시간이 너무 오래 걸린다 !
- Resizing 과정에서 **이미지의 손실, 왜곡**이 일어날 수 있고, 이는 학습을 저해하는 요인으로 작용할 수 있다 !
- SS, CNN, Classifier 총 3개의 모델을 각각 따로 학습시켜야 한다 !

SPP-Net (간단히)

- 전체 Image를 **한번만 CNN에 넣어** feature map 구하고
그 값들을 proposal들을 계산할 때 공유
- Input image를 FC Layer에 통과할 때, Resizing하지 않고
Pooling 통해 고정 길이 벡터로 만듦

SPP-Net 의 한계

- CNN을 먼저 거친 후, proposal 추출하므로 proposal로 구한 Loss값이 앞의 CNN의 back-propagation에 큰 영향을 미치지 못함
- 여전히 3개의 모델(SS, CNN, Classifier) 을 각각 학습시켜야 함

지금까지의 문제

- 3개의 모델 (SS, CNN, Classifier)을 각각 학습시켜야 한다 !
- 아직도 Train, Test 시간이 충분히 빠르지 않다 !

Fast R-CNN

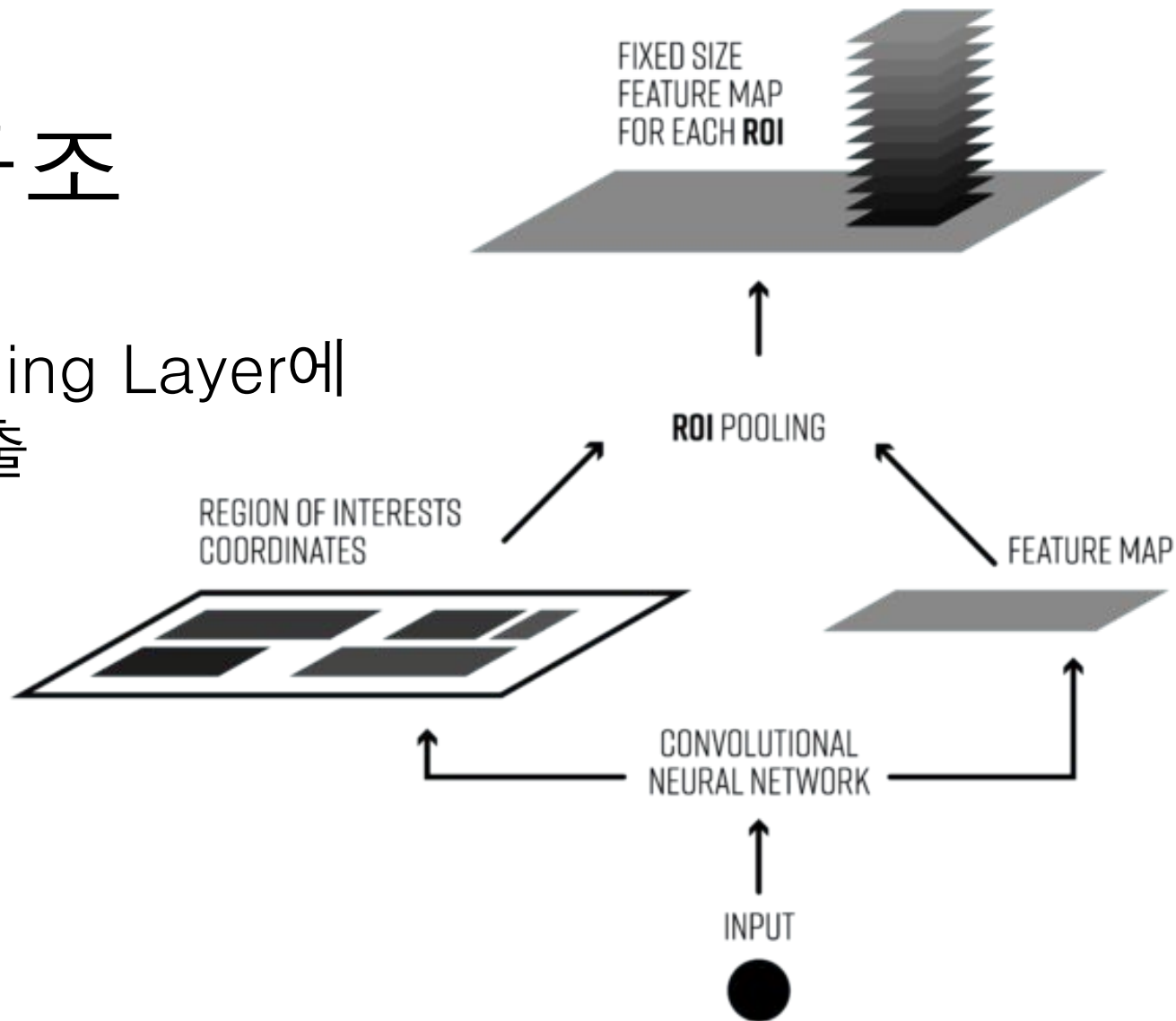
- 마이크로소프트 연구팀에서 제시한 Object Detection 모델
- **단일 모델로 만들어 End-to-end 학습**
- 기존에 제시된 R-CNN, SPP-Net보다 성능도 우수하고, 훈련 및 테스트 시간이 굉장히 개선됨

Fast R-CNN 의 구조

1. SS를 통해 2,000개의 proposal의 **정보**만 먼저 구함
(정보: proposal의 시작점 좌표, 너비, 높이)
2. 전체 image를 CNN에 넣음. 이때, CNN의 Input, Output size
가 달라도 그 비율과 미리 구해 놓은 proposal의 정보로
Output feature map에서 proposal의 위치를 알 수 있음

Fast R-CNN 의 구조

3. 각 proposal들을 RoI Pooling Layer에 넣어 고정 길이 벡터로 추출



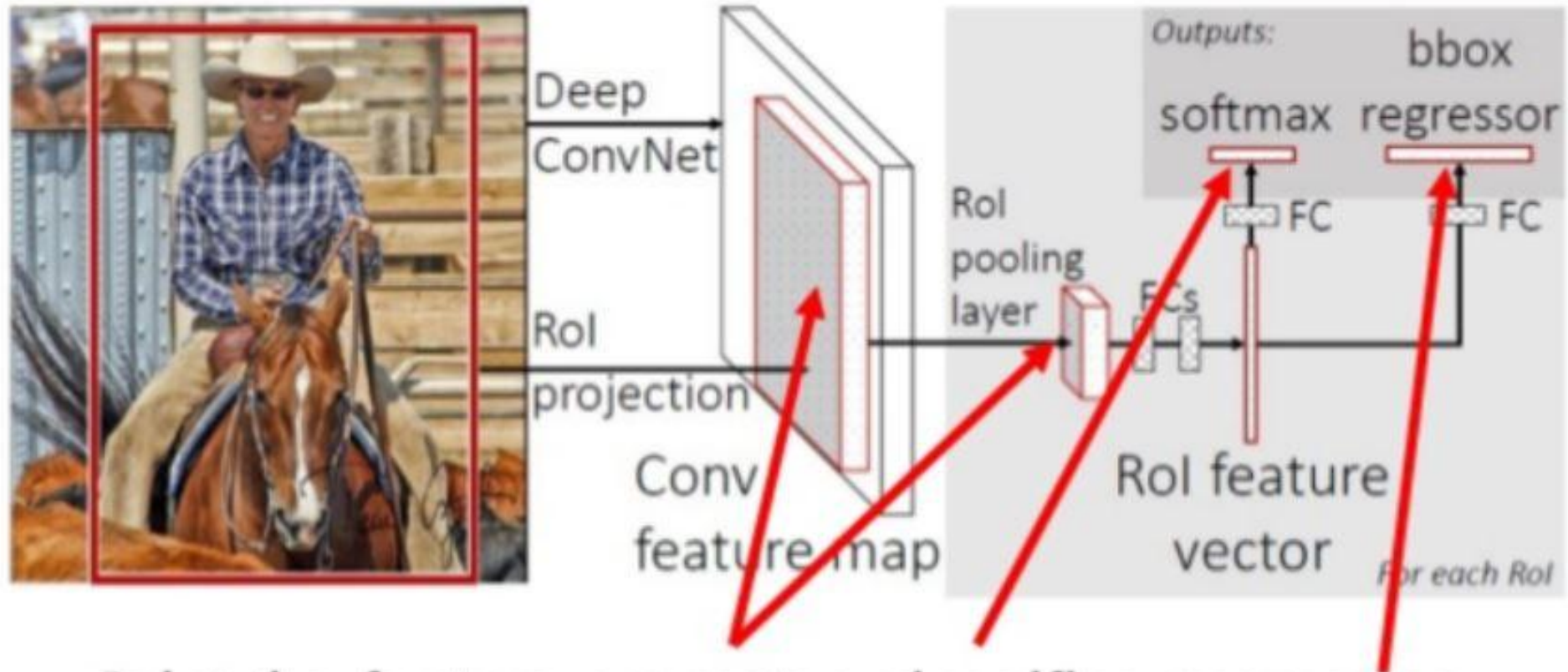
4. 추출된 벡터를 Classifier FC layer와 Bounding-box regressors

FC layer에 벡렬전으로 넣으

Fast R-CNN 의 구조

- RoI(Region of Interest) Pooling Layer: Proposal을 넣어 output이 7x7로 나오도록 pooling하는 layer. SPP-Net에서 아이디어를 차용했으며, 1층 짜리 SPP layer와 같음
- Classifier fc layer: 기존 SVM Classifier를 Neural network Softmax로 만들어 network에 합류시킴
- Bounding-box regressors fc layer: 4개의 value를 출력 이는 proposal의 정보로써, 조금 더 tight하게 boxing하는 역할

Fast R-CNN: Joint Training Framework



Joint the feature extractor, classifier, regressor together in a unified framework

Fast R-CNN 의 구현

- 입력으로는 Image와 해당 Image의 Roi목록을 받음
- ImageNet으로 pre-trained된 VCG모델을 기본으로 사용
- 마지막 Pooling layer 대신 Roi pooling layer가 들어간다.
- Fc layer는 Classifier(softmax)와 bounding-box regressor, 2개로 나눠짐
- Mini-batch를 사용하고, 하나의 batch 당 2개의 image, image당 64개의 Roi가 사용됨

Fast R-CNN 의 구현

- Loss function

$$L(p, u, t^u, v) = L_{cls}(p, u) + \lambda[u \geq 1]L_{loc}(t^u, v)$$

- L_{cls} : K개의 object 와 1개의 background로 K+1개의 class를 갖는 Softmax Log Loss
- $L_{loc}(t^u, v)$: Class별로 bounding-box를 추정하는 L1 Loss로 구성된 Regressor Loss

Fast R-CNN 의 성능

- R-CNN 대비 평균 5% 정도 성능 향상이 있었음
- R-CNN 대비 train 속도 84시간 -> 8시간
- R-CNN 대비 test 속도 49초 -> 2초

Fast R-CNN 의 한계

- Test 시간 2초는 아직 Real Time으로 적용할 수 없다는 반증
→ SS(2초), CNN + Classification(0.5초)로 대부분의 시간은 SS
가 소요함
- 이후, Selective Search 과정 또한 딥러닝으로 해결한
Faster R-CNN이 나옴



감사합니다